

Procesadores de Lenguajes, Grado de Ingeniería Informática



Enunciado de la práctica obligatoria

Jaime Urquiza Fuentes

En este documento se especifica el enunciado de la práctica obligatoria de la asignatura de Procesadores de Lenguajes (Grado de Ingeniería Informática y sus respectivas titulaciones dobles). En concreto, tanto la parte obligatoria como la opcional del analizador léxico, sintáctico y la traducción dirigida por la sintaxis, así como las fechas de entrega de las diferentes fases de la práctica.



Práctica obligatoria

Procesadores de Lenguajes

Tabla de contenidos

Práctica obligatoria	2
Procesadores de Lenguajes	2
Introducción	3
Material de entrega.....	3
Calificación.....	3
Plazos de entrega	4
Especificación de la práctica	4
Parte obligatoria.....	5
Especificaciones léxicas del lenguaje fuente.....	5
Especificación sintáctica del lenguaje fuente.....	6
Especificación de la traducción dirigida por la sintaxis	8
Parte opcional.....	11
Especificación léxica del lenguaje fuente	11
Especificación sintáctica del lenguaje fuente.....	12
Especificación de la traducción dirigida por la sintaxis	13

Introducción

La práctica se podrá realizar en grupos de, como máximo, 3 personas. La puntuación obtenida no depende del número de integrantes del grupo, tampoco tiene por qué ser igual para todos los integrantes.

No se permite la integración de personas en un grupo después de la primera entrega.

Sí se permite la salida de personas de un grupo, dejando claro en una entrevista con el profesor, quién continúa con la práctica y quién no hace la práctica o decide empezar una nueva.

Material de entrega

Una **memoria escrita** en formato electrónico que incluya:

- Un informe del trabajo realizado, así como cualquier anotación o característica que se desee hacer notar, **sin incluir listados fuente**. Específicamente debe incluir:
 - Una breve descripción de reglas léxicas que se crean de especial relevancia.
 - Demostración del procedimiento realizado para transformar la gramática a **LL(I)**.
 - Se debe aportar al menos un ejemplo relevante de la práctica.
 - Se deben aportar los conjuntos directores en un anexo.
 - Explicación de los errores contenidos en los 4 casos de prueba aportados (ver siguientes puntos)
 - Muy breve descripción de la implementación de la notificación de errores (si se opta al notable)
 - Muy breve descripción de la resolución de la recuperación de errores (si se opta al sobresaliente)
- 8 casos de prueba de los cuales, 4 han de ser correctos y 4 erróneos, de forma que permitan observar el comportamiento del procesador.
- Recordad: **LO BUENO, SI BREVE, DOS VECES BUENO**

Aplicación informática que implemente la funcionalidad requerida para la entrega correspondiente (léxico, sintáctico o completa):

- Ejecutable de la aplicación, que debe funcionar sobre **plataforma Windows 10 disponible en MyApps**. Este debe ser un fichero JAR que reciba como argumento la ruta al fichero de entrada y se ejecute por terminal mediante la siguiente interfaz: `java -jar miPrograma.jar ruta_fichero.txt`.
- Proyecto de desarrollo completo incluyendo listados fuente de las especificaciones e implementación.
- Los ficheros asociados a los casos de prueba que aparecen en la memoria.

La calidad del material entregado es responsabilidad de los estudiantes. En caso de encontrar una entrega con virus o defectuosa de forma que no pueda ser evaluada será considerada suspensa.

Calificación

La calificación de la práctica se divide en tres niveles:

- **aprobado** (hasta 5), completando la parte obligatoria.
- **notable** (hasta 7), alcanzando el grado de aprobado, proporcionando recuperación de errores y completando los siguientes aspectos de la parte opcional: constantes en base binaria, octal y hexadecimal; las constantes y operadores lógicos NOT, AND y OR; las sentencias de control de flujo IF en todas sus formas y DO WHILE.

- **sobresaliente** (hasta 9,5), alcanzando el grado de notable y notificando los errores en español de forma detallada (línea, columna y posible causa) así como completando toda la parte opcional.

Además, se otorgará **medio punto extra** en función de la **calidad de la memoria final**.

Plazos de entrega

- **7 de Abril de 2026** - Analizadores léxico y sintáctico. Evaluación ordinaria.
- **21 de Mayo de 2026** - Práctica completa. Evaluación ordinaria.
- **Junio de 2026** - Práctica completa. Evaluación extraordinaria.

Especificación de la práctica

La práctica consiste en el **diseño e implementación de un traductor de programas** escritos en un lenguaje de programación **de cálculo científico** (de ahora en adelante **lenguaje fuente**), a otro **similar a C** (de ahora en adelante **lenguaje final**).

Parte obligatoria

Especificaciones léxicas del lenguaje fuente

Los elementos del lenguaje que aparecen entrecomillados en la gramática (que se muestra en la especificación), deben aparecer **tal cual** (sin las dobles comillas) en cualquier programa correctamente escrito en este lenguaje, el resto de elementos se especifican a continuación.

Los **identificadores**, representados por el símbolo `IDENT` de la gramática, son ristra de símbolos compuestas por letras (del alfabeto inglés, por lo tanto ni “ñ” ni “Ç”), dígitos (en base decimal) y guiones bajos “_” (underscore). Empiezan obligatoriamente por una letra. Ejemplos correctos: `contador`, `contador1`, `acumulador_total_2`. No se permite utilizar como identificadores las palabras reservadas del lenguaje.

Las **constantes numéricas** pueden ser de **dos tipos**: enteras y reales. Están representadas en la gramática por los símbolos terminales `NUM_INT_CONST` y `NUM_REAL_CONST`, respectivamente. Todas las ristra de dígitos (uno o más dígitos) a las que se hace referencia a continuación se especifican en base decimal:

- Las constantes numéricas **enteras** son una ristra de dígitos, opcionalmente precedida del signo “-” en caso de ser negativas.
- Las constantes numéricas **reales** pueden opcionalmente ir precedidas del signo “-” (en caso de ser negativas) y se pueden expresar de tres formas distintas:
 - Punto fijo: dos ristra de dígitos separadas por el punto decimal.
 - Exponencial: una ristra de dígitos seguida del carácter “e” o “E”, un signo “-” opcional (en caso de que el exponente sea negativo) y otra ristra de dígitos.
 - Mixto: que sería una constante real en punto fijo seguida del carácter “e” o “E”, un signo “-” opcional (en caso de que el exponente sea negativo) y otra ristra de dígitos.

Ejemplos de constantes correctamente escritas:

- Enteras: `123`, `-690`, `5`, `000078`, `-005`, `00`
- Reales:

Punto fijo		Exponencial		Mixto	
Constante	Valor	Constante	Valor	Constante	Valor
<code>123.456</code>	<code>123.456</code>	<code>123E456</code>	<code>123 x 10⁴⁵⁶</code>	<code>1.23E4</code>	<code>1.23 x 10⁴</code>
<code>-00.69</code>	<code>-0.69</code>	<code>-64e-77</code>	<code>-64 x 10⁻⁷⁷</code>	<code>-000.64E-77</code>	<code>-0.64 x 10⁻⁷⁷</code>
<code>45.07000</code>	<code>45.07</code>	<code>-045e6</code>	<code>-45 x 10⁶</code>	<code>-045.0e16</code>	<code>-45.0 x 10¹⁶</code>
<code>-123.456</code>	<code>-123.456</code>	<code>003E-35</code>	<code>3 x 10⁻³⁵</code>	<code>0.03E-35</code>	<code>0.03 x 10⁻³⁵</code>

Las **constantes literales**, representadas en la gramática por el símbolo terminal `STRING_CONST`, son ristra de símbolos delimitadas por comillas simples: `'contenido de la constante literal'` o comillas dobles: `"contenido de la constante literal"`. El contenido de las

constantes puede ser cualquier carácter que pueda aparecer en el programa fuente. Si se desea que algún símbolo delimitador aparezca como contenido existen dos opciones:

- Delimitar con un símbolo cuando se quiera que aparezca el otro como contenido.
- Duplicar el símbolo que se quiere que aparezca en el contenido cuando éste es el usado como delimitador de la constante.

Constante	Valor
'comilla doble " dentro'	comilla doble " dentro
"comilla simple ' dentro"	comilla simple ' dentro
'comilla simple '' dentro'	comilla simple ' dentro
"comilla doble "" dentro"	comilla doble " dentro
'comilla doble " y simple '' dentro'	comilla doble " y simple ' dentro
"comilla simple ' y doble "" dentro"	comilla simple ' y doble " dentro

Los **comentarios** se especifican comenzando con el carácter **!** y terminando con el fin de línea. Por lo tanto no existen comentarios de multiples líneas como en C, Java, Pascal, etc.

Especificación sintáctica del lenguaje fuente (

Un programa está compuesto por cuatro partes: la zona de declaraciones de constantes y variables (dcllist), la zona de declaración de funciones y procedimientos (cabecera), la zona de sentencias del programa principal (sentlist) y la zona de implementación de funciones y procedimientos (subproglis).

```

prg ::= "PROGRAM" IDENT ";" dcllist cabecera sentlist "END"
      "PROGRAM" IDENT subproglis
dcllist ::=  $\Lambda$  | dcl dcllist
cabecera ::=  $\Lambda$  | "INTERFACE" cablist "END" "INTERFACE"
cablist ::= decproc decsubprog | decfun decsubprog
decsubprog ::=  $\Lambda$  | decproc decsubprog | decfun decsubprog
sentlist ::= sent | sentlist sent

```

La primera zona de declaraciones es una lista de declaraciones de constantes y variables, estas últimas con opción a inicializarlas.

```

dcl ::= defcte | defvar
defcte ::=  $\Lambda$ 
          | tipo "," "PARAMETER" ":@" IDENT "=" simpvalue ctelist ";"
          defcte
ctelist ::=  $\Lambda$  | "," IDENT "=" simpvalue ctelist
simpvalue ::= NUM_INT_CONST | NUM_REAL_CONST | STRING_CONST

defvar ::=  $\Lambda$  | tipo ":@" varlist ";" defvar
tipo ::= "INTEGER" | "REAL" | "CHARACTER" charlength
charlength ::=  $\Lambda$  | "(" NUM_INT_CONST ")"
varlist ::= IDENT init | varlist "," IDENT init
init ::=  $\Lambda$  | "=" simpvalue

```

La segunda zona de declaraciones es una lista de cabeceras de procedimientos y/o funciones. Los dos tienen una estructura similar al programa principal.

```
decproc ::= "SUBROUTINE" IDENT formal_paramlist
        dec_s_paramlist "END" "SUBROUTINE" IDENT
formal_paramlist ::=  $\wedge$  | "(" nomparamlist ")"
nomparamlist ::= IDENT | IDENT "," nomparamlist
dec_s_paramlist ::=  $\wedge$  | tipo "," "INTENT" "(" tipoparam ")" IDENT ";"
        dec_s_paramlist

tipoparam ::= "IN" | "OUT" | "INOUT"
decfun ::= "FUNCTION" IDENT "(" nomparamlist ")"
        tipo ":" IDENT ";"
        dec_f_paramlist
        "END" "FUNCTION" IDENT
dec_f_paramlist ::=  $\wedge$ 
        | dec_f_paramlist tipo "," "INTENT" "(" "IN" ")"
        IDENT ";"
```

La zona de sentencias del programa principal es una lista de sentencias como asignaciones y llamadas a procedimientos:

```
sent ::= IDENT "=" exp ";" | proc_call ";"
exp ::= exp op exp | factor
op ::= oparit
oparit ::= "+" | "-" | "*" | "/"
factor ::= simpvalue | "(" exp ")" | IDENT "(" exp explist ")" | IDENT
explist ::= "," exp explist |  $\wedge$ 
proc_call ::= "CALL" IDENT subpparamlist
subpparamlist ::= "(" exp explist ")" |  $\wedge$ 
```

La zona de implementación de funciones y procedimientos consiste en una lista de estas con sus correspondientes cabeceras.

```
subproglis ::= codproc subproglis | codfun subproglis |  $\wedge$ 
codproc ::= "SUBROUTINE" IDENT formal_paramlist
        dec_s_paramlist
        declist sentlist
        "END" "SUBROUTINE" IDENT
codfun ::= "FUNCTION" IDENT "(" nomparamlist ")"
        tipo ":" IDENT ";"
        dec_f_paramlist
        declist sentlist
        IDENT "=" exp ";"
        "END" "FUNCTION" IDENT
```

Tanto en la declaración de subprogramas (decproc y decfun) como en su implementación (codproc y codfun) la declaración de parámetros formales (dec_s_paramlist y dec_f_paramlist) se debe realizar en el mismo orden que se mencionan en la cabecera (nomparamlist)

Tanto en esta parte como en la parte opcional hay que asegurarse de que la **gramática usada con ANTLR es LL(1)** y se especifica en **notación BNF**. Además, **no está permitido el uso del patrón Visitor**.

Especificación de la traducción dirigida por la sintaxis

El objetivo es traducir el código en lenguaje fuente a un código en lenguaje final, cuya gramática proporcionaremos más adelante. El código en lenguaje final estará compuesto por un solo fichero de extensión `.c` y cuyo nombre será el mismo que el del fichero de entrada. Así, el fichero de prueba "ejemplo.for" producirá como salida el fichero "ejemplo.c".

Los comentarios que se encuentren en el lenguaje fuente deben ser ignorados (no aparecerán en el lenguaje final). Por otro lado, tanto los identificadores como los operadores aritméticos y las constantes numéricas (`CONSTINT` y `CONSTFLOAT`) se expresarán en el lenguaje final de forma **idéntica** a como aparecen en el lenguaje fuente. Esto no ocurre de forma completa con otros elementos que se detallan a continuación:

- Palabras reservadas sobre tipos de datos básicos como "INTEGER" y "REAL", que aparecerán en el lenguaje final como "int" y "float" respectivamente.
- Las declaraciones de variables de tipo cadena de caracteres con "CHARACTER" se especificarán en el lenguaje final como "char" y si van acompañadas de longitud, esta se especificará entre corchetes "[" y "]" en el lenguaje final. Se puede ver un ejemplo más adelante en la traducción de sentencias de declaración.
- Las constantes literales del lenguaje final (`CONSTLIT`) son rstras de símbolos delimitadas únicamente por comillas dobles: "constante literal". Si se necesita que el contenido contenga las comillas dobles, habrá que escaparlas con la barra inclinada "\", por ejemplo: "constante literal con \" en el contenido".

La gramática del lenguaje final se presenta a continuación. Un programa está compuesto por tres partes, la declaración de constantes (representada en la gramática por el símbolo `defines`), la declaración de las cabeceras de las funciones (representada en la gramática por el símbolo `decfuncs`) y un conjunto de sucesivas implementaciones completas de funciones (representadas en la gramática por el símbolo `partes`).

```
program ::= defines decfuncs partes
```

```
defines ::=  $\Lambda$  | "#define" IDENT ctes defines
```

```
ctes ::= CONSTINT | CONSTFLOAT | CONSTLIT
```

```
decfuncs ::=  $\Lambda$  | type restdecfun decfuncs
```

```
type ::= "void" | typevar
```

```
restdecfun ::= IDENT "(" listparam ")" ";"  
| IDENT "(" "void" ")" ";"
```

```
typevar ::= "char" | "int" | "float"
```

```
listparam ::= listparam "," type IDENT dim | type IDENT dim
```

```
dim ::=  $\Lambda$  | "[" "]"
```

```
partes ::=  $\Lambda$  | part partes
```

```
part ::= type restpart
```

```
restpart ::= IDENT "(" listparam ")" blq  
| IDENT "(" "void" ")" blq
```

```
blq ::= "{" sentlist "}"
```

En el lenguaje final, la declaración de constantes se hace **únicamente al comienzo del programa**, independientemente de dónde estén en el programa fuente, y según se especifica en la gramática correspondiente en el símbolo `defines`. Cada declaración de constante en el lenguaje final aparecerá

en una línea nueva. Es decir, no pueden aparecer varias sentencias “#define” en la misma línea. A continuación, se puede ver un ejemplo.

Entrada fuente	Salida en lenguaje final
<pre> INTEGER , PARAMETER :: bloque = 1024; REAL , PARAMETER :: PI = 3.141592, E = 2.718281; CHARACTER (2) , PARAMETER :: S = "SI", N = "NO"; </pre>	<pre> #define bloque 1024 #define PI 3.141592 #define E 2.718281 #define S "SI" #define N "NO" </pre>

Además, como se ha dicho, en el lenguaje final sólo hay funciones. Así, la zona de cabecera del lenguaje fuente está representada por el símbolo *decfuncs* de la gramática del lenguaje final. A continuación, se muestra un ejemplo de cabecera, nótese que:

- En esta parte obligatoria el tipo de acceso de parámetros (IN, OUT o INOUT) no se traduce al lenguaje final.
- En los parámetros de tipos de cadena de caracteres no se generan dimensión, solo los corchetes indicando que es una cadena.

Entrada fuente	Salida en lenguaje final
<pre> INTERFACE ! Declaracion de funcion CON parametros FUNCTION fun1 (a, b) INTEGER :: fun1; INTEGER, INTENT (IN) a; CHARACTER(4), INTENT (IN) b; END FUNCTION fun1 ! Declaracion de procedimiento CON parametros SUBROUTINE proc1 (c, d, e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; END SUBROUTINE proc1 ! Declaracion de procedimiento SIN parametros SUBROUTINE proc2 END SUBROUTINE proc2 END INTERFACE </pre>	<pre> int fun1 (int a , char b[]); void proc1 (float c , int d, int e); void proc2 (void); </pre>

La zona de implementación de funciones y procedimientos del lenguaje fuente (símbolo gramatical *subproglst*) está representada por el símbolo *partes* de la gramática del lenguaje final. Como se ha visto en las cabeceras, los procedimientos del lenguaje fuente se declararán funciones del lenguaje final cuyo tipo devuelto es “void”. Así, en el lenguaje final, la diferencia entre procedimientos y funciones es que los primeros devuelven el tipo “void”, mientras que las segundas devuelven el tipo “int”, “float” o “char”. Del mismo modo, el uso de funciones se especifica en la gramática correspondiente en los símbolos *sent* y *factor*. A continuación, se muestran unos ejemplos de declaración-implementación y uso de estos elementos.

Entrada fuente	Salida en lenguaje final
<pre> !Implementacion de procedimiento SIN parametros SUBROUTINE proc2 !... ! Llamada a procedimiento con parametros CALL proc1 (c, d, e); !... END SUBROUTINE proc2 </pre>	<pre> void proc2 (void) { ... proc1(c,d,e); ... } </pre>

Entrada fuente	Salida en lenguaje final
<pre>!Implementacion de procedimiento CON parametros SUBROUTINE proc1 (c, d , e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; !... ! llamada a funcion con parametros c = fun1(d , 1.0); !... END SUBROUTINE proc1</pre>	<pre>void proc1 (float c , int d, int e){ ... c = fun1(d , 1.0); ... }</pre>

Además, en el lenguaje fuente, para devolver un valor como resultado de una función se usan sentencias de asignación. Estas sentencias tienen una particularidad: el identificador en la parte izquierda de la asignación es el nombre de la función dentro de la que están ubicadas. En el lenguaje final serán sustituidas por sentencias de retorno que comienzan por la palabra reservada “*return*” seguida de la expresión existente en la parte derecha de la sentencia de asignación del lenguaje fuente.

Entrada fuente	Salida en lenguaje final
<pre>!Implementacion de funcion CON parametros FUNCTION fun1 (a, b) INTEGER :: fun1; INTEGER, INTENT (IN) a; REAL, INTENT (IN) B; !... ! Llamada a procedimiento ! sin parametros CALL proc2; !... ! Valor devuelto por fun1 fun1 = a * 2; END FUNCTION fun1</pre>	<pre>int fun1 (int a , float b){ ... proc2(); ... return a * 2; }</pre>

En el lenguaje fuente, las sentencias del programa principal lo conforman aquellas sentencias, símbolos *sent* y *sentlist*, que se encuentran inmediatamente antes de las palabras reservadas *END PROGRAM*. En el lenguaje final, el programa principal se declarará como un procedimiento cuyo nombre es “*main*” que además no tendrá parámetros formales. Así, las sentencias del programa principal del lenguaje fuente irán dentro de este procedimiento en el lenguaje final.

```
void main ( void )
{
  ...
}
```

Pasamos a ver la parte de la gramática del lenguaje final correspondiente a las sentencias, símbolos *sent* y *sentlist*, contenidas tanto dentro del programa principal como de funciones y procedimientos. Además de las sentencias que ya hemos visto sobre retorno de funciones y llamadas a funciones y procedimientos; también se pueden encontrar sentencias de declaración de variables y asignación.

```
sentlist ::= sentlist sent | sent
sent ::= type lid ";" | IDENT "=" exp ";" | IDENT "(" lexp ")" ";"
       | IDENT "(" ")" ";" | "return" exp ";"
lid ::= IDENT dims init | lid "," IDENT dims init
dims ::=  $\lambda$  | "[" CONSTANT "]"
init ::=  $\lambda$  | "=" ctes
lexp ::= exp | lexp "," exp
```

```
exp ::= exp op exp | factor
op  ::= "+" | "-" | "*" | "/"
factor ::= IDENT "(" lexp ")" | IDENT "(" " " ")"
        | "(" exp ")" | IDENT | ctes
```

La traducción de las sentencias de asignación se circunscribe a las especificaciones léxicas de traducción ya descritas, puesto que como se puede ver, la sintaxis es igual en el lenguaje fuente y final. Respecto a las sentencias de declaración, deben seguirse las especificaciones léxicas y sintácticas de traducción ya descritas, a continuación se ofrecen algunos ejemplos:

Entrada fuente	Salida en lenguaje final
<pre>INTEGER :: var1, var2 = 24; REAL :: var3; CHARACTER :: var4, var5 = "-", var6; CHARACTER (10):: var7, var8;</pre>	<pre>int var1, var2 = 24; float var3; char var4, var5 = "-", var6; char var7[10], var8[10];</pre>

Las variables declaradas dentro del programa principal en el lenguaje fuente aparecerán al comienzo del procedimiento “main” del lenguaje final. Todas las funciones y procedimientos declarados dentro del programa principal en el lenguaje fuente aparecerán como funciones y procedimientos independientes en el lenguaje final, siempre declarados antes del procedimiento “main”. Finalmente, las variables declaradas en cada función o procedimiento del lenguaje fuente aparecerán al comienzo dentro de su correspondiente función o procedimiento del lenguaje final.

Para terminar con la parte obligatoria, el código en lenguaje final generado tendrá la siguiente propiedad en cuanto a tabulación se refiere: todas las sentencias entre los símbolos “{” y “}”, se colocarán en un nivel de tabulación mayor (indentación a la derecha) que estos.

Parte opcional

En esta parte se explican las ampliaciones de la parte obligatoria, estas ampliaciones consistirán en nuevos tokens a reconocer, que, junto con nuevas construcciones sintácticas, habrá que integrar en la gramática de la parte obligatoria. Ambos, los nuevos tokens y las nuevas construcciones sintácticas provocarán también nuevos requisitos de traducción dirigida por la sintaxis. Todo esto se describe en esta sección. De forma general, las ampliaciones en la gramática se describirán aumentando las producciones existentes e incluyendo otras nuevas. Así la producción:

$A ::= \dots \mid D$

Significa que al símbolo A ya existente en la gramática se le añade (este es el significado de los $\dots$) una nueva producción $A ::= D$, manteniendo todas las que tenía en la gramática de la parte obligatoria.

Especificación léxica del lenguaje fuente

Constantes enteras positivas en base binaria, octal y hexadecimal

Las constantes numéricas enteras positivas también se podrán especificar en la base binaria (NUM_INT_CONST_B), octal (NUM_INT_CONST_O) y hexadecimal (NUM_INT_CONST_H). Cumplen el mismo papel que las constantes enteras en cuanto a tipo de valores. Cada una de estas serán ristra de dígitos de la base correspondiente, entre comillas simples, sin signo (siempre son positivas) y precedidas de la letra “b”, “o” o “z” según sean respectivamente binarias, octales o hexadecimales. A continuación se ofrecen ejemplos de constantes binarias, octales y hexadecimales correcta e incorrectamente escritas:

Correctas	Incorrectas
Binaria : b'011'	b'017'
Octal : o'740'	b'910'
Octal : o'101'	b'1D1'
Hexadecimal : z'A34'	b'11G'
Hexadecimal : z'890'	o'830'
Hexadecimal : z'106'	o'6E0'
Hexadecimal : z'010'	o'42Z'
	z'8AH5'

Constantes lógicas

Las **constantes lógicas** representan los valores de verdad: cierto `.TRUE.` y falso `.FALSE.`. Nótese que los puntos que rodean a las palabras `TRUE` y `FALSE` también forman parte de la constante.

Especificación sintáctica del lenguaje fuente

Constantes enteras positivas en base binaria, octal y hexadecimal

Las constantes numéricas enteras positivas en base binaria (`NUM_INT_CONST_B`), octal (`NUM_INT_CONST_O`) y hexadecimal (`NUM_INT_CONST_H`), se incluyen en la gramática con las siguientes reglas:

```
simpvalue ::= ... | NUM_INT_CONST_B | NUM_INT_CONST_O | NUM_INT_CONST_H
```

Sentencias de control de flujo

Las sentencias de control de flujo se basan en la comprobación de expresiones condicionales (nótese que incluyen las constantes lógicas para los valores cierto y falso) cuya gramática es:

```
expcond ::= expcond oplog expcond | factorcond
oplog   ::= ".OR." | ".AND." | ".EQV." | ".NEQV."
factorcond ::= exp opcomp exp | "(" expcond ")" | ".NOT." factorcond
           | ".TRUE." | ".FALSE."
opcomp  ::= "<" | ">" | "<=" | ">=" | "==" | "/="
```

La gramática del lenguaje fuente para las sentencias de control de flujo es:

```
sent ::= ...
      | "IF" "(" expcond ")" sent
      | "IF" "(" expcond ")" "THEN" sentlist "ENDIF"
      | "IF" "(" expcond ")" "THEN" sentlist "ELSE" sentlist "ENDIF"
      | "DO" "WHILE" "(" expcond ")" sentlist "ENDDO"
      | "DO" IDENT "=" doval "," doval "," doval sentlist "ENDDO"
      | "SELECT" "CASE" "(" exp ")" casos "END" "SELECT"
doval ::= NUM_INT_CONST | IDENT

casos ::= "CASE" "(" etiquetas ")" sentlist casos
        | "CASE" "DEFAULT" sentlist
        | ^

etiquetas ::= simpvalue listaetiquetas
           | simpvalue ":" simpvalue
           | ":" simpvalue
           | simpvalue ":"

listaetiquetas ::= "," simpvalue listaetiquetas | ^
```

Especificación de la traducción dirigida por la sintaxis

A continuación, se detallan los requisitos en cuanto a la generación del código en lenguaje final correspondiente a la parte opcional de la práctica.

Constantes enteras positivas en base binaria, octal y hexadecimal

Este tipo de constantes estará representado en el **lenguaje final** por el símbolo *CONSTANT*. El formato de estas constantes en el lenguaje final es un prefijo que identifica a la base, seguido de los dígitos de la constante. Los prefijos serán "0b" para la base binaria, "0o" para la base octal y "0x" para la base hexadecimal. A continuación se ofrecen un conjunto de ejemplos:

Leguaje fuente	Leguaje final
b'011'	0b011
o'740'	0o740
o'101'	0o101

Leguaje fuente	Leguaje final
z'A34'	0xA34
z'890'	0x890
z'106'	0x106
z'010'	0x010

Constantes lógicas

El lenguaje final no tiene constantes lógicas específicas, en su lugar se usan números enteros donde el valor 0 es equivalente a *.FALSE.* y cualquier valor distinto de 0, por ejemplo 1, es equivalente a *.TRUE.*

Sentencias de control de flujo

Como se explicaba anteriormente, las sentencias de control de flujo se basan en la comprobación de expresiones condicionales. La gramática del lenguaje final de estas expresiones es:

```
lcond ::= lcond opl lcond | cond | "!" cond
opl  ::= "|" | "&&" | "^" | "!"^
cond  ::= exp opr exp | "(" cond ")" | CONSTANT
opr   ::= "==" | "<" | ">" | ">=" | "<=" | "!="
```

La equivalencia de operadores se detalla en las siguientes tablas:

Lenguaje fuente	Lenguaje Final
.OR.	
.AND.	&&
.NOT.	!

Lenguaje fuente	Lenguaje Final
.EQV.	!^
.NEQV.	^
/=	!=

La gramática del lenguaje final para las sentencias de control de flujo es la siguiente:

```
sent ::= ...
      | "if" "(" lcond ")" blq
      | "if" "(" lcond ")" blq "else" blq
      | "while" "(" lcond ")" blq
      | "for" "(" IDENT "=" exp ";" lcond ";" IDENT "=" exp ")" blq
      | "switch" "(" exp ")" "{" casos "}"

casos ::= "case" etiqs ":" sentlist casos
      | "case" etiqs ":" casos
      | "case" "default" sentlist
      | ^

etiqs ::= ctes | ctes "to" ctes | ">" ctes | ctes "<"
```

La traducción de las sentencias de bifurcación (IF) y bucle mientras (DO-WHILE) es directa según la gramática antes mencionada. A continuación, se muestran varios ejemplos-esquema:

Entrada fuente	Salida en lenguaje final
IF (condicion) una-sentencia;	if (condicion) { una-sentencia; }
IF (condicion) THEN Sentencias ENDIF	if (condicion) { Sentencias }
IF (condicion) THEN Sentencias-then ELSE Sentencias-else ENDIF	if (condicion) { Sentencias-then } else { Sentencias-else }
DO WHILE (condicion) Sentencias ENDDO	while (condicion) { Sentencias }

La sentencia “DO” es equivalente al bucle “para”. El identificador, **vble** en el ejemplo, hace de contador, especificando el valor inicial (**val-ini**), el límite para la condición de continuidad (**val-limite**) y el incremento (**inc**). Se traduce de forma similar a una sentencia “for” en C, donde la condición de continuidad del bucle es la desigualdad del identificador con el valor límite. El esquema de traducción es el siguiente:

Entrada fuente	Salida en lenguaje final
DO vble = val-ini , val-limite , inc Sentencias ENDDO	for (vble=val-ini ; vble!=val-limite ; vble=vble+inc) { Sentencias }

Finalmente, la sentencia de selección de múltiples casos es la sentencia “SELECT CASE”, que se traduce a la sentencia “switch-case” según los siguientes esquema ejemplo:

Entrada fuente	Salida en lenguaje final
! Ejemplo con un caso simple, un ! caso multiple con tres casos ! integrados y un caso por defecto SELECT CASE (selector) CASE (caso0) Sentencias-caso-1 CASE (caso1 , caso2 , caso3) Sentencias-caso-2 CASE DEFAULT Sentencias-caso-def END SELECT	switch (selector) { case caso0 : Sentencias-caso-1 break; case caso1 : case caso2 : case caso3 : Sentencias-caso-2 break; default: Sentencias-caso-def }

Entrada fuente	Salida en lenguaje final
<pre>! Ejemplo con un caso de rango, ! caso con límite superior y ! caso con límite inferior SELECT CASE (selector) CASE (caso0 : caso1) Sentencias-caso-1 CASE (: caso2) Sentencias-caso-2 CASE (caso3 :) Sentencias-caso-3 END SELECT</pre>	<pre>switch (selector) { case caso0 to caso1: Sentencias-caso-1 break; case < caso2 : Sentencias-caso-2 break; case > caso3 : Sentencias-caso-3 break; }</pre>

Nótese que todos los casos, excepto el correspondiente al caso por defecto, terminan con una sentencia “break;”.

Traducción de parámetros OUT e INOUT de procedimientos

Este tipo de parámetros está pensado para implementar el paso de parámetros por referencia, es decir que cambien su valor dentro del procedimiento y mantengan ese valor actualizado una vez termine la ejecución del procedimiento. El lenguaje final no tiene este mecanismo, pero se puede simular mediante el uso de punteros de la forma que se explicará a continuación (acompañada por el esquema-ejemplo):

- En lo que se refiere a la cabecera de la función resultante, los parámetros de paso por referencia serán declarados como punteros al tipo especificado en la cabecera del código en lenguaje fuente: “int *”, “float *” o “char *”.

Entrada fuente	Salida en lenguaje final
<pre>INTERFACE ... SUBROUTINE procl (c, d, e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; END SUBROUTINE procl ... END INTERFACE</pre>	<pre>... void procl (float *c, int d, int *e); ...</pre>

- El uso de esos parámetros dentro de la implementación de la función siempre irá acompañado del operador de referencia “*”. Para acceder al parámetro “h”, tanto en lectura como escritura, se escribirá “*h”.

Entrada fuente	Salida en lenguaje final
<pre>SUBROUTINE procl (c, d, e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; ... c = expresion ; ... e = otra-expresion ; ... END SUBROUTINE procl</pre>	<pre>void procl (float *c, int d, int *e){ ... *c = expresion ; ... *e = otra-expresion ; ... }</pre>

- En la llamada a la función, habrá que indicar que se proporciona la dirección de la variable enviada como parámetro mediante el operador de dereferencia "&". Para enviar la variable "h" como parámetro por referencia se escribirá "&h".

Entrada fuente	Salida en lenguaje final
... proc1 (param1, param2, param3); ...	... proc1 (¶m1, param2, ¶m3) ; ...

Para ello la gramática del lenguaje final tiene las siguientes reglas añadidas:

```
typevar ::= ... | "char" "*" | "int" "*" | "float" "*"
sent ::= ... | "*" IDENT "=" exp ";"
factor ::= ... | "&" IDENT | "*" IDENT
```

Como es lógico, los parámetros reales pasados por referencia deben ser variables, no expresiones, pero esa comprobación **no es necesaria que se realice** en esta práctica.

Comprobación de nombres en funciones y procedimientos

Las funciones y procedimientos necesitan hacer varias comprobaciones en cuanto a los identificadores utilizados en su declaración e implementación.

En primer lugar, tanto en la declaración como en la implementación de procedimientos y funciones se especifica un identificador al principio y al final. Dichos identificadores deben coincidir.

```
SUBROUTINE IDENT ... END SUBROUTINE IDENT
FUNCTION IDENT ... END FUNCTION IDENT
```

En segundo lugar, tanto en las funciones como en los procedimientos y tanto en su declaración de cabecera como en su implementación, los nombres de los parámetros entre paréntesis en la cabecera deben coincidir con los nombres de los parámetros en la especificación de su tipo.

Declaración de cabecera	Implementación
INTERFACE SUBROUTINE proc1 (c, d, e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; END SUBROUTINE proc1 END INTERFACE	SUBROUTINE proc1 (c, d, e) REAL, INTENT (OUT) c; INTEGER, INTENT (IN) d; INTEGER, INTENT (INOUT) e; ! ... Contenido del procedimiento END SUBROUTINE proc1

En tercer lugar, en el caso de las funciones, el nombre de la función debe aparecer en la cabecera, asociado al tipo devuelto por la función y finalmente, como parte izquierda de la asignación del valor devuelto por la función, la última sentencia de asignación del cuerpo de la función.

Declaración de cabecera	Implementación
INTERFACE FUNCTION fun1 (a, b) INTEGER :: fun1; INTEGER, INTENT (IN) a; CHARACTER(4), INTENT (IN) b; END FUNCTION fun1 END INTERFACE	FUNCTION fun1 (a, b) INTEGER :: fun1; INTEGER, INTENT (IN) a; CHARACTER(4), INTENT (IN) b; !... ! Contenido de la función fun1 = a * 2; END FUNCTION fun1

En caso de detectar el incumplimiento de alguna de estas condiciones se debe notificar el error y no generar ninguna salida salvo las notificaciones de esos errores.